Appendix: Artifact Evaluation

Artifact Evaluation (AE)

Paper ID: pap142

This appendix evaluates the three computational artifacts defined in the AD:

- A1: TempGNN reference model, tests, and edge-stream profiling.
- A2: four-system U280 forward-path execution and measurement workflow.
- A3: source-labeled paper-reference CSV/SVG reconstruction.

The recommended reviewer order is A1 -> A3 -> A2. The first two paths do not

require FPGA access. Artifact A2 uses an author-provided Alveo U280 account;

credentials are delivered only through the private SC submission channel.

I. Evaluation Scope and Badge Boundary

The artifact is prepared to support Artifacts Available and Artifacts

Evaluated-Functional. Apache-2.0 is included; a version-specific DOI must be

present by the August 25, 2026 artifact freeze.

The current package does not assert Results Reproduced. The fresh U280 path

uses bounded 8,192-event prefixes, 8-dimensional Q10 kernels, deterministic

stand-in weights, and xbutil total-board power. The paper uses complete model

configurations, default 32-bit floating point, trained checkpoints, full

evaluation streams, and post-route Vivado power estimates. The configuration

therefore records results_reproduced_eligible: false. Packaged reference rows

are never substituted for a failed fresh measurement.

Expected direct review time, excluding optional Vitis rebuilds:

Artifact	Setup, execution, and analysis time
A1	5-10 min setup; less than 2 min execution; less than 2 min analysis
A2	10-20 min setup; 30-90 min execution; less than 5 min analysis
A3	less than 5 min setup; less than 1 min execution; 2-5 min analysis

II. Computational Artifact A1

A. Artifact Setup

Use Linux x86_64 with Python 3.10 or newer, GNU Make, and Bash:

```
git clone https://github.com/TempGNN-Program/␣
↪ TempGNN.git
cd TempGNN
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

The frozen package may be used instead of git clone:

```
tar -xzf pap142_tempgnn_sc26_ae_u280.tgz
cd pap142_tempgnn_sc26_ae
```

No FPGA is needed. Unit fixtures are generated locally. Optional Q14 profiling

downloads TGL edge streams; packaged U280-prefix samples are already under

external/u280_dataset_samples/ with URL and SHA256 metadata.

B. Artifact Execution

The task dependency chain is:

```
A1-T1 environment
  -> A1-T2 unit tests for TDP/DDTC/OATS and
  ↪ artifact contracts
  -> A1-T3 optional real-edge acquisition
  -> A1-T4 optional edge-stream profiling
  -> A1-T5 CSV/Markdown summary generation
```

Run the required fast path:

```
python3 -m unittest discover -s tests
```

Expected terminal summary:

```
Ran 34 tests
OK
```

Run the optional real-edge path:

```
make data
make q14
```

The default Q14 parameters are WIKI/MOOC/REDDIT, four models, fanout 20, depth

2, and the batch parameters recorded in each output row. make data is skipped

when the corresponding edge files are already present.

C. Artifact Analysis

All 34 tests must pass. In particular, the recursive TDP state must agree with

chronological updates, fixtures must be deterministic, xclbin link requests

must agree with the Vivado-connected clock and nonnegative post-route WNS/TNS,

and duplicate xclbins must be rejected by preflight.

Optional Q14 outputs:

```
results/q14_real_tgl_edges/q14_dataset_model_␣
↪ summary.csv
results/q14_real_tgl_edges/q14_batches.csv
results/q14_real_tgl_edges/q14_summary.md
```

Inspect packet hit/reuse rates, collisions, stalls, and memory traffic. The

latency columns in this CPU path use a 225 MHz cycle model and are not fresh

U280 board timings. Passing A1 demonstrates that the mechanisms supporting

contributions C1–C3 are executable and internally checked.

III. Computational Artifact A2

A. Artifact Setup

Hardware and recorded software environment:

```
Board: AMD/Xilinx Alveo U280,
↪  xcu280-fsvh2892-2L-e
Platform: xilinx_u280_gen3x16_xdma_1_202211_1
OS: Ubuntu 22.04 x86_64
XRT: 2.16.204
Vitis/Vivado: 2023.2 (rebuild only)
GCC: 11.4.0
GNU Make: 4.3
Python: 3.10 or newer
```

The reviewer receives a dedicated, time-bounded account through the SC private

channel. After login:

```
source /opt/xilinx/xrt/setup.sh
cd /path/to/pap142_tempgnn_sc26_ae
xrt-smi examine
```

The four metadata-normalized artifacts are:

```
artifacts/u280/TempGNN/bin/tempgnn_forward_ke⌋
↪  rnel.hw.xclbin
artifacts/u280/MATG/bin/matg_kernel.hw.xclbin
artifacts/u280/ViTeGNN/bin/vitegnn_kernel.hw.⌋
↪  xclbin
artifacts/u280/RTGA/bin/rtga_kernel.hw.xclbin
```

Each validated XRT host and the baseline C-sim reference binaries are colocated

under the same `bin/` directories. TempGNN uses its 21-CU parallel host;

the baselines use the common single-CU host. Source, paper-mechanism mapping, and limitations

are in `hardware/baselines/README.md` and `hardware/baselines/MECHANISM_MAP.md`.

The repository and frozen package also include the exact real-input fixture

metadata and goldens used by the packaged run under `results/generated_u280_comparison_fixtures/`.

Optional rebuilds require:

```
source
↪  /tools/Xilinx/Vitis/2023.2/settings64.sh
```

Rebuilding is not required for evaluation and can take several hours per

xclbin. The direct-run path stays within the SC26 review budget.

B. Artifact Execution

The workflow is implemented by `scripts/run_u280_core_reproduction.py`:

```
A2-T1 preflight four source revisions, hosts,
↪  and distinct xclbin hashes
  -> A2-T2 generate fixtures from bundled
  ↪  real-data prefixes
  -> A2-T3 generate independent software
  ↪  goldens
  -> A2-T4 load and run each implementation on
  ↪  U280
  -> A2-T5 gate the repeated-kernel window and
  ↪  sample total board power
  -> A2-T6 validate checksums, repetitions,
  ↪  energy, provenance, and clocks
  -> A2-T7 aggregate raw rows and derive
  ↪  measured core comparison tables
  -> A2-T8 compare fresh tables with the
  ↪  paper-reference rows
```

Run preflight:

```
make u280-core-preflight
```

Expected: U280 four-implementation preflight PASS, followed by four different

xclbin SHA256 values.

Run the complete direct measurement:

```
make ae-core-u280 U280_CORE_DEVICE=0
↪  U280_CORE_REPETITIONS=3
```

The default-device wrapper is `bashscripts/run_all.shu280-core`.

Checked parameters are loaded from `configs/u280_core_reproduction.json`:

```
datasets: WK, MC, RT, LM, WT, GT
models: JODIE, TGN, TGAT, APAN
implementations: TempGNN, MATG, ViTeGNN, RTGA
real prefix: 8,192 events per dataset
target batch: 1,000
repetitions: 3
tensor path: 8-dimensional Q10
requested clocks: TempGNN 168 MHz;
↪  MATG/ViTeGNN/RTGA 225 MHz
achieved-clock comparison tolerance: 0.5 MHz
```

Each dataset/model pair is calibrated to a repeated-kernel measurement window.

The host performs a warmup, waits at a file gate, and then executes the measured

iterations while the harness samples `xbutil examine --report electrical`.

Optional one-fixture TempGNN sanity command:

```
make u280-run U280_DEVICE=0
```

Optional rebuild commands:

```
make u280-build \
  U280_PLATFORM=/path/to/xilinx_u280_gen3x16_⌋
  ↪  xdma_1_202211_1.xpfm
make u280-baseline-build
make u280-stage-artifacts
```

Baseline links should be run serially on a 64 GB host. The staging command

redacts login, host, and absolute paths from textual evidence and the xclbin

`BUILD_METADATA`/`SYSTEM_METADATA` sections; it verifies that the FPGA

`BITSTREAM` section is byte identical before and after normalization.

C. Artifact Analysis

The run creates a new directory and never overwrites paper-reference data:

```
results/reviewer_u280_runs/<run-id>/provenanc⌋
↪  e.json
results/reviewer_u280_runs/<run-id>/raw/TempG⌋
↪  NN/measurements.csv
results/reviewer_u280_runs/<run-id>/raw/MATG/⌋
↪  measurements.csv
```

```
results/reviewer_u280_runs/<run-id>/raw/ViTeG
↪ NN/measurements.csv
results/reviewer_u280_runs/<run-id>/raw/RTGA/
↪ measurements.csv
results/reviewer_u280_runs/<run-id>/baselines
↪ _u280/
results/reviewer_u280_runs/<run-id>/derived_c
↪ omparison_figures/
results/reviewer_u280_runs/<run-id>/verificat
↪ ion.json
results/reviewer_u280_runs/<run-id>/verificat
↪ ion.md
```

The packaged final run is `results/reviewer_u280_runs/20260717T024537Z/`; its four raw CSV files contain 72 rows each (288 total), with zero golden, repeat, or timing failures. The fresh all-workload averages are `1.296553 ms` for TempGNN and `9.966618 ms` for MATG, yielding **7.8889x** measured average speedup. The paper-tolerance diagnostic remains FAIL and is preserved.

For 3 repetitions, each implementation must contain 72 rows and the combined

matrix must contain 288 rows. Every row must report:

- `golden_validation=PASS` and `repeat_consistency=PASS`;
- a nonzero kernel and embedding checksum;
- a real dataset source URL and 64-character input/golden hashes;
- positive latency and power;
- energy consistent with `latency_ms * power_w`;
- requested and xclbin-link-requested clocks, the Vivado-connected post-route kernel clock, WNS/TNS, and `timing_met=PASS`.

The workflow refuses normalized comparison generation if an implementation's

post-route clock changes between rows or lacks timing closure. Each actual

clock remains in the raw rows and no frequency rescaling is applied.

`verification.md` reports the maximum relative error against

paper-reference Fig.11 and the U280 subset of Fig.12. The reviewer Make target

default one-command path preserves a tolerance FAIL while still completing a

functionally valid hardware run. `make ae-core-u280-strict` makes that

mismatch fail the command. It must not be converted to PASS by adjusting measured rows.

Passing `A2` establishes that four distinct implementations execute on the

same U280, produce stable software-checked outputs, and emit traceable raw

measurement data. The bounded reproduction scope prevents this path from independently

supporting a Results Reproduced claim for `C4`.

IV. Computational Artifact A3

A. Artifact Setup

Use the Python environment created for `A1`. No FPGA, GPU, external dataset, or

network access is required. Inputs are:

```
tempgenn/paper_reference_data.py
reference_inputs/README.md
```

The source module has 668 structured records. Every record identifies its source class, locator, and

uncertainty. Original author workbooks and vector files are not distributed

because they contain author metadata; only the anonymized numeric extraction

and source hashes are embedded in code. The AE archive also includes deterministic CSV/SVG outputs under `results/paper_reproduction/` for direct inspection; the code-embedded records remain their authoritative source.

B. Artifact Execution

The dependency chain is:

```
A3-T1 validate the code-embedded records and
↪ source labels
  -> A3-T2 split records by figure
  -> A3-T3 generate matching CSV and SVG files
  -> A3-T4 generate the combined table and
  ↪ manifest
  -> A3-T5 run consistency tests
  -> A3-T6 regenerate the reviewer report
```

Commands:

```
python3 -m scripts.reproduce_paper_figures
python3 -m scripts.make_ae_report \
  --q14-summary results/q14_real_tgl_edges/q1
  ↪ 4_dataset_model_summary.csv \
  --board-json results/board_u280/summary.json
  ↪ \
  --out results/ae_report
```

Equivalent Make target:

```
make smoke
make report
```

`make smoke` includes the 34 unit and artifact-contract tests from `A1-A3`.

C. Artifact Analysis

The following outputs are included for direct inspection and regenerated in place by the commands above:

```
results/paper_reproduction/paper_figure_value
↪ s.csv
results/paper_reproduction/all_figure_data.csv
results/paper_reproduction/figure_data_manife
↪ st.csv
results/paper_reproduction/fig2_execution_bre
↪ akdown.csv/.svg
results/paper_reproduction/fig4a_branch_paral
↪ lelism_ratio.csv/.svg
results/paper_reproduction/fig9b_gpu_overhead
↪ _breakdown.csv/.svg
results/paper_reproduction/fig10_speedup_tgli
↪ te_cpu.csv/.svg
results/paper_reproduction/fig11_speedup_matg
↪ .csv/.svg
```

```
results/paper_reproduction/fig12_energy_tempg
↪  nn.csv/.svg
results/paper_reproduction/fig13_ablation_tim
↪  e.csv/.svg
results/paper_reproduction/fig14a_batch_sensi
↪  tivity.csv/.svg
results/paper_reproduction/fig14b_tdp_entries
↪  .csv/.svg
```

Expected summary checks:

```
TempGNN vs TGLite-CPU: 132.80x
TempGNN-G vs TGLite-CPU: 10.85x
Cascade vs TGLite-CPU: 5.22x
TempGNN vs MATG, explicit plotted AVG: 7.7889x
Paper prose for TempGNN vs MATG: 7.6x
Energy, Cascade/TempGNN, explicit plotted AVG:
↪  33.545x
w/o DDTC normalized time: 3.08x
w/o OATS normalized time: 1.77x
```

The Fig.11 explicit AVG and the paper's rounded prose value are both preserved.

The generator must not alter individual values to force agreement. Generated

SVG titles contain `Paper-reference`, and the manifest identifies whether each

series came from exact workbook cells or axis-calibrated vector geometry.

Passing `A3` demonstrates deterministic reconstruction and provenance of the

code-embedded paper plotting records for `C4`. It does not demonstrate a fresh rerun

of MATG, ViTeGNN, RTGA, Cascade, TGLite-CPU, or the complete TempGNN paper

configuration.

V. Badge Evidence Summary

Badge	Current evidence
Artifacts Available	Source, tests, inputs, CSV/SVG records, four metadata-normalized U280 xclbins, reports, and documentation; Apache-2.0 included and DOI required by freeze
Artifacts Evaluated-Functional	`A1` tests, baseline C-sim, four-xclbin preflight, board goldens, repeated outputs, power/latency rows, and post-route evidence
Results Reproduced	Not currently asserted because `A2` is a bounded mechanism-level comparison rather than the paper's full precision, checkpoints, streams, and power method